화면과 그 뒤의 데이터 흐름까지 확인하는 프론트엔드 개발자

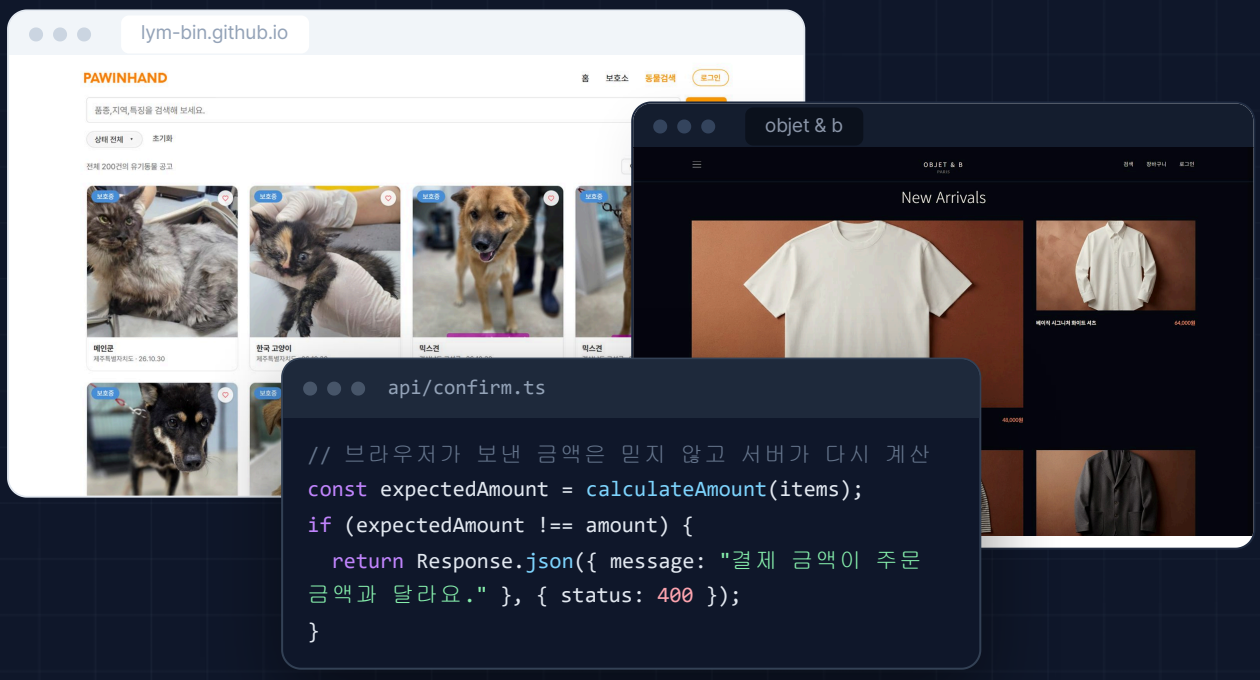
바닐라 JavaScript로 기본기를 다시 세우고, React와 TypeScript로 결제까지 이어지는 쇼핑몰을 만들었습니다. 추측보다 측정으로, 로컬보다 배포 환경과 실기기에서 확인합니다.

606 → 242 KB
포인핸드 API 응답 크기

14 → 7 초
포인핸드 로드 완료 시간

8 개
결제 금액 계산 단위 테스트

2 개
운영 중인 개인 프로젝트



임상빈
Frontend Developer

010-5150-5589 · persie24@naver.com
sangbin-dev.vercel.app · github.com/lym-bin · velog.io/@bin-dev

기본기부터 다시 쌓은 프론트엔드 개발자 ABOUT ME

동작하는 코드에서 멈추지 않고, 왜 그렇게 동작하는지와 다른 환경에서도 같은지를 확인합니다.

걸어온 길

Java/Spring 기반 SI 프로젝트에서 레거시 JSP를 MVC 구조로 재구성하고 Nexacro 화면에 백엔드 로직을 연동하며, 백엔드가 화면과 맞물리는 구조를 먼저 경험했습니다. 그 뒤 사용자와 가장 가까운 화면 쪽으로 방향을 바꿨습니다.

프레임워크보다 기본기를 먼저 세우려고 바닐라 JavaScript로 공공 API 서비스를 만들었고, 그 과정에서 느낀 한계를 해결하려고 React와 TypeScript로 결제까지 이어지는 쇼핑몰을 만들었습니다.

기술 스택

주력

HTML · CSS · JavaScript · React · TypeScript · Tailwind CSS

함께 사용

Firebase · Next.js · Vite · React Router · React Query · Vitest · GSAP · Vercel Functions · 토스페이먼츠 · Git · Figma

Next.js(App Router)는 이 포트폴리오 사이트와, Objet & B 상품 목록을 Next.js로 다시 만든 학습 프로젝트(objet-b-next)에서 사용했습니다.

확인하는 습관

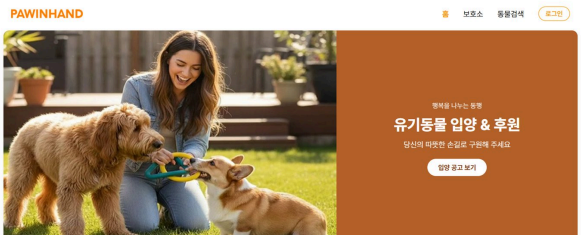
01 · MEASURE
추측 말고 측정
느린 원인을 짐작하지 않고 Network 탭으로 응답 크기와 시간을 잽니다.

02 · PREVIEW
머지 전 미리보기 배포
PR마다 실제 서버 환경에서 먼저 확인해, 로컬에서 안 보이던 문제를 잡습니다.

03 · DEVICE
실기기 확인
PC의 기기 모드로는 안 잡히는 문제를 아이폰 실기기에서 찾았습니다.

04 · RECORD
이유를 기록
PR에 변경 이유와 확인 결과를 남기고, 큰 문제는 Velog에 정리합니다.

프로젝트

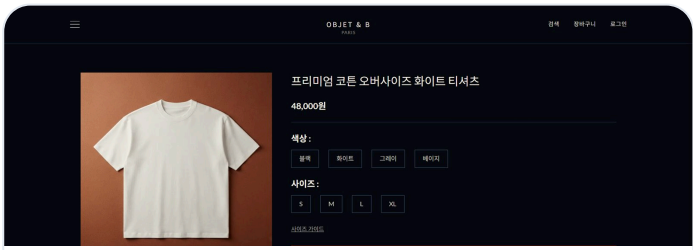


PROJECT 01 · VANILLA JS

포인핸드 웹사이트 리디자인

공공데이터 API로 만든 유기동물 입양 정보 서비스. 프레임워크 없이 API 레이어, 공통 UI 상태, 캐시를 직접 설계

2026.06 – 09 · 개인 · HTML/CSS/JS · GitHub Pages



PROJECT 02 · REACT

Objet & B 미니멀 쇼핑몰

React + TypeScript 쇼핑몰. 토스페이먼츠 결제를 서버 금액 검증까지 연결하고, Vitest와 React Query를 도입

2026.07 – 10 · 개인 · React/TS/Firebase · Vercel

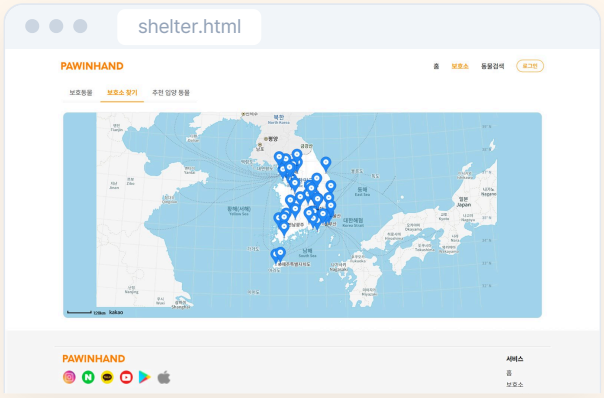
VANILLA JAVASCRIPT

포인핸드 웹사이트 리디자인

유기동물 입양 플랫폼 '포인핸드'를 레퍼런스로, 공공데이터 API를 연동해 검색 · 필터 · 지도 · 상세까지 프레임워크 없이 구현한 비공식 리디자인

기간	역할
2026.06 – 2026.09	1인 기획 · 디자인 · 개발
스택	**배포**
HTML · CSS · JS(ES6+)	GitHub Pages

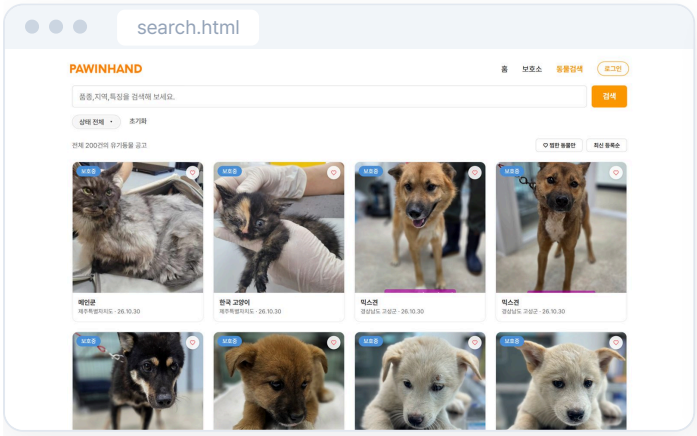
Live GitHub 스타일가이드



01 페이지와 주요 기능 FEATURES

8개 페이지를 직접 마크업하고, 페이지마다 스크립트를 나눠 필요한 기능만 불러오도록 했습니다.

페이지	주요 기능
index	히어로 배너, 지역별 추천 동물 필터(API의 `orgNm` 집계), 베스트 후기, 소식 슬라이더, 유기동물 현황 미니 대시보드
search	키워드 실시간 검색(품종 · 지역 · 특징), 상태 필터, 정렬, 더보기 페이지네이션, 찜하기(localStorage)
detail	공고번호(`?num=`) 기반 상세 — sessionStorage 캐시 우선, 없으면 목록 조회로 대체. 이미지 갤러리, 댓글(localStorage)
shelter	추천 입양 동물, 보호동물 목록, 조건 필터 모달, 카카오맵 보호소 마커(주소 → 좌표 지오코딩)
stats	처리 결과별 마리 수, 시도별 안락사율 막대 그래프와 '표로 보기' 토글 (공공데이터 통계 API)
news	캠페인 · 이벤트 소식
login / signup	ID 중복 · 비밀번호 8자 규칙 · 확인 일치 검증, 인라인 오류 표시, 체험 계정, localStorage 세션 (백엔드 없는 학습용 구현)



search — 검색 · 상태 필터 · 정렬 · 찜



stats — 요약 타일과 HTML/CSS 막대 그래프

02 구조 설계 ARCHITECTURE

프레임워크가 없어도 역할을 나누면 관리할 수 있다고 보고, 데이터 요청 · 공통 UI · 페이지 로직을 파일 단위로 분리했습니다.



API 레이어 — 실패해도 화면이 안 깨지게

요청 · 응답 확인 · 파싱을 `api.js`에 모았습니다. 실패하면 예외 대신 빈 배열을 돌려줘서, 호출하는 쪽은 "데이터가 없음"만 처리하면 되고 `Promise.all`로 묶은 요청도 한꺼번에 실패하지 않습니다.

js/api.js (발췌)

```
async function fetchAnimalsList(numOfRows = 200) {
  try {
    const response = await fetch(url);
    if (!response.ok) throw new Error(`HTTP
${response.status}`);
    const data = await response.json();
    return data?.response?.body?.items?.item || [];
  } catch (error) {
    return []; // 실패해도 호출한 쪽이 안 터짐
  }
}
```

이벤트 위임 — 다시 그려도 리스너는 하나

검색 · 필터 때마다 카드 목록을 다시 그리기 때문에, 카드마다 리스너를 붙이지 않고 목록 부모에 하나만 붙였습니다. 찜 상태는 localStorage에 공고번호 배열로 저장합니다.

js/search.js (발췌)

```
// 찜 버튼 — 이벤트 위임
list.addEventListener("click", (e) => {
  const btn = e.target.closest(".like-btn");
  if (!btn) return;
  const nowLiked = toggleLike(btn.dataset.num);
  btn.textContent = nowLiked ? "♥" : "♡";
  btn.classList.toggle("liked", nowLiked);
});
```

03 상세 페이지 데이터 흐름 DATA FLOW

공공 API는 공고번호로 한 건만 조회하는 기능이 없습니다. 처음엔 상세 페이지마다 목록 1,000건을 받아 `find()`로 찾았는데, 목록에서 이미 받은 데이터를 다시 받는 낭비였습니다.

목록에서 상세로 이동 요청 0회

- **목록 페이지**가 받은 원본을 `cacheAnimals()` 로 sessionStorage에 공고번호 키로 저장
- 카드 클릭 → `detail.html?num=…`
- 상세 페이지가 **캐시를 먼저 확인** → 있으면 바로 렌더

새로고침 · 주소 직접 입력 대체 경로

- 캐시에 해당 공고번호가 없음
- 목록 API로 1,000건 조회 후 `find()`
- 그래도 없으면 화면 안에 "찾을 수 없음" 안내와 검색으로 돌아가는 링크

js/detail.js (발췌)

```
let currentAnimal = getCachedAnimal(targetNum);   // 1) 캐시 먼저
if (!currentAnimal) {                             // 2) 없을 때만 목록 조회
  const apiItems = await fetchProtectData({ numOfRows: 1000 });
  currentAnimal = apiItems.find((item) => String(item.desertionNo) === String(targetNum));
}
```

04 UI 상태 설계 LOADING · EMPTY · ERROR

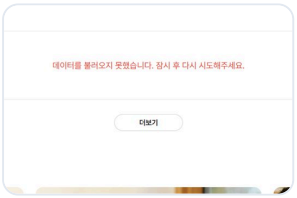
외부 API와 지도에 의존하는 서비스라, 요청이 느리거나 실패할 때의 화면을 `ui.js` 공통 함수로 한 곳에서 관리했습니다.

showSkeleton()

로딩

첫 카드만 `role="status"` 로 스크린리더에 안내, 나머지는 `aria-hidden` . 모션 줄이기 설정이면 애니메이션 끔

showStateMessage()



빈 결과 · 에러

alert 대신 목록 자리에 안내 문구. 빈 결과와 에러를 다른 스타일로 구분

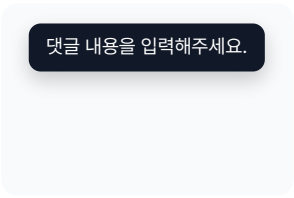
지도 대체



지도 실패

SDK 로드를 25회(약 5초) 기다려도 안 되거나 좌표 변환이 모두 실패하면 보호소 텍스트 목록으로

showToast() · imgError()



토스트 · 이미지 실패

화면을 막는 alert 대신 2.5초 뒤 사라지는 알림. 깨진 이미지는 회색 대체 이미지로

로딩 · 에러 · 지도 대체는 배포된 사이트에서 API 응답을 늦추거나 막아 실제로 재현해 캡처한 화면입니다. 토스트는 실제 문구로 다시 그린 그림입니다.

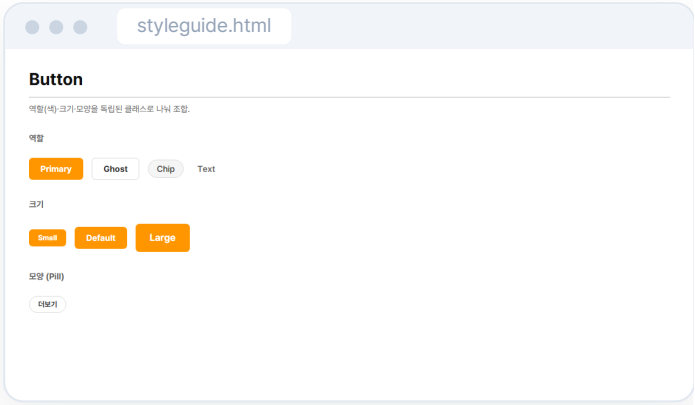
05 디자인 토큰과 컴포넌트 DESIGN SYSTEM IN CODE

색상 · 글자 크기 · 간격 · 모서리 · 그림자 · 레이어 순서를 `common.css` 의 `:root` 변수로 정의하고, 규칙을 스타일가이드 페이지로 문서화했습니다.

Primary	Text Strong	Text Sub	보호중	긴급	종료/입양
--color-primary	--color-text-strong	--color-text-sub	--color-state-protect	--color-state-urgent	--color-state-done
#ff9900	#111111	#666666	#4a90d9	#e74c3c	#27ae60

css/common.css (발췌)

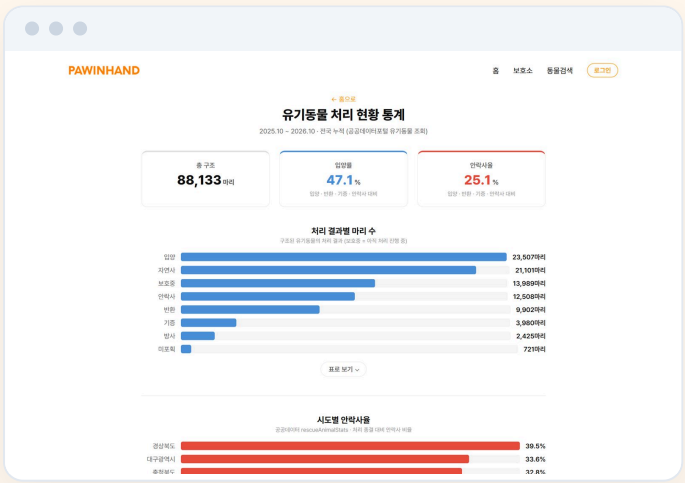
```css
:root {
  --font-hero: 2rem;      /* 32px */
  --font-body: 1rem;      /* 16px */
  --space-md: 20px;      /* 카드 */
  --space-xl: 70px;      /* 섹션 */
  --radius-md: 12px;
  --container-max: 1200px;
  --z-modal: 1000;
  --z-toast: 2000;
}
```



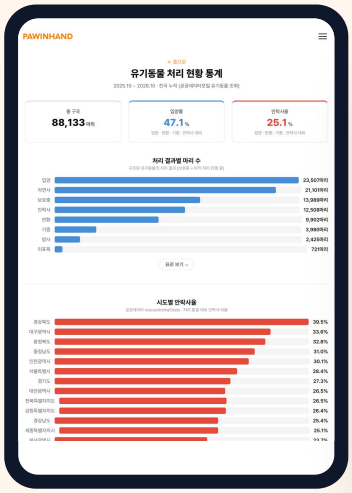
.btn 버튼 시스템 — 역할(primary · ghost · chip · text) · 크기 · 모양을 독립 클래스로 조합. 페이지마다 제각각이던 버튼을 하나로 통합

06 반응형 RESPONSIVE

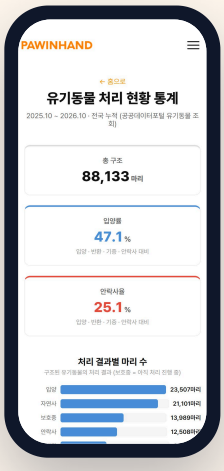
Desktop-first로 `max-width: 1024px` · `768px` 두 지점에서 레이아웃을 바꿉니다. 같은 통계 페이지를 세 해상도로 확인했습니다.



Desktop · 1440px



Tablet · 900px



Mobile · 390px

07 성능 개선 — 추측 말고 측정으로 PERFORMANCE

목록 API로 한 번에 500건을 받으면서 첫 화면이 늦게 떴습니다. 그럴듯한 최적화부터 적용했지만 크게 줄지 않았고, 측정해 보니 원인은 다른 곳에 있었습니다.

먼저 시도한 것

- △ 무거운 메인 배너 SVG를 PNG로 바꿔 압축
- △ 순서대로 보내던 요청을 `Promise.all` 병렬 요청으로
- △ 이미지에 `loading="lazy"` 적용
- ✓ **Network 탭 측정** → 병목은 요청 방식이 아니라 **한 번에 받는 데이터 양**(`numOfRows=500`). 첫 화면에 필요한 200건으로 줄임

결과 (Network 탭 실측)

API 응답 크기 · 개선 전 **606KB**

개선 후 (약 60% 감소) **242KB**

로드 완료 시간 · 개선 전 **14초**

개선 후 **7초**

08 트러블슈팅 TROUBLESHOOTING

KAKAO MAPS SDK

보호소 지도가 에러 없이 먹통

원인 `autoload=false`라 `kakao.maps.services`는 `load()` 콜백 이후에 생기는데, 대기 조건에서 그걸 검사해 `load()`를 영영 호출하지 못하고 재시도만 반복

해결 대기 조건을 `kakao.maps.load` 존재 여부로 바꾸고, 재시도 25회(약 5초)를 넘으면 보호소 텍스트 목록으로 대체

PUBLIC API · STATS

통계 수치가 비율이 아니라 마리 수

원인 통계 API의 `tot`를 %로 표시했는데 실제로는 처리 상태별 마리 수. 기본값 `numOfRows=10`이라 항목 일부가 잘림

해결 `numOfRows=50`으로 모든 항목을 받고, 비율은 전체 합계로 직접 계산

SECURITY · XSS

댓글에 스크립트가 실행될 수 있었음

원인 사용자가 쓴 댓글을 `innerHTML` 템플릿으로 그려서 `<img onerror>` 같은 입력이 실행될 수 있음

해결 `createElement` + `textContent`로 바꿔 입력이 항상 글자로만 그려지게

RESPONSIVE

모바일에서 가로 스크롤

원인 푸터 4단 메뉴(`.nav-list`)가 좁은 화면에서 줄바꿈 없이 펼쳐져 화면 폭을 63px 넘김

해결 768px 이하에서 `flex-wrap`으로 2열로 접히게 수정

NETWORK

공공 API가 갑자기 응답하지 않음

원인 콘솔엔 오류가 없었고 Network 탭에서 요청이 pending으로 멈춤. 트래픽 · 인증키를 의심했지만 원인은 API 주소의 http/https 불일치

해결 엔드포인트를 https로 수정

LAYOUT

반응형에서 이미지가 찌그러짐

원인 고정 height 때문에 폭이 줄어도 높이가 그대로라 이미지가 눌림

해결 고정 높이 대신 `aspect-ratio`로 원본 비율 유지

09 설계 판단과 회고 DECISIONS · RETROSPECTIVE

DECISION

백엔드 없는 로그인을 어디까지 만들까

서버가 없어 회원 정보를 localStorage에 저장했습니다. 비밀번호는 일부러 해시하지 않았습니다. 브라우저에만 저장되는 값이라 해시해도 보안상 의미가 없고, 오히려 "실제 서비스라면 서버 · HTTPS · 단방향 해시가 전제"라는 점을 분명히 하기 위해서입니다. 가입 검증과 계정 대조까지만 만들고, 토큰 발급 같은 실제 인증은 재현하지 않았습니다.

DECISION

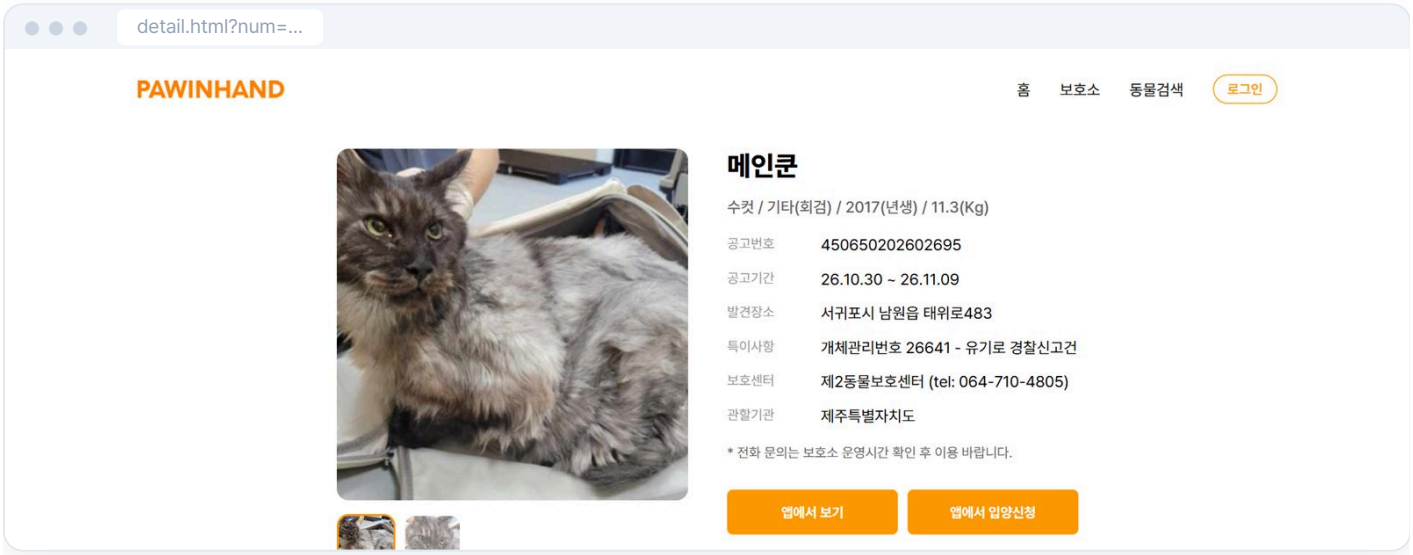
API 키를 저장소에 둔 이유

GitHub Pages는 빌드 단계가 없어 배포할 때 키를 주입할 수 없습니다. 사용한 키는 공공데이터 조회용과, 도메인 제한을 건 카카오 지도용이라 포트폴리오 운영 기간 동안 노출을 감수하기로 하고 README에 이유를 밝혔습니다.

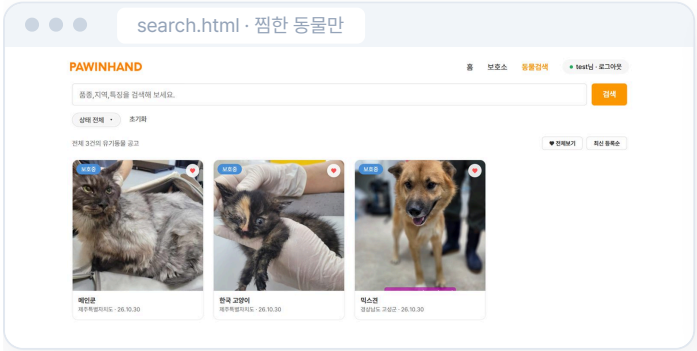
RETROSPECTIVE

React를 선택하게 된 이유

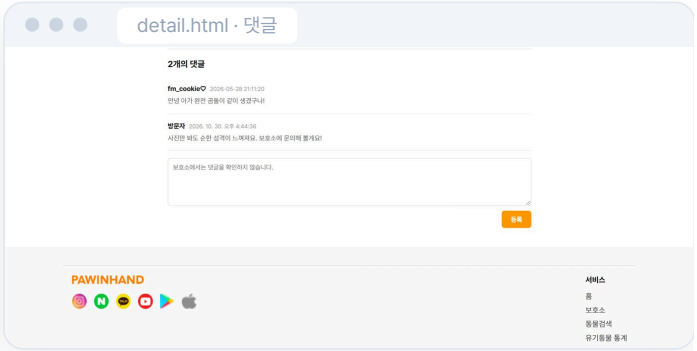
데이터가 바뀔 때마다 `innerHTML`로 목록을 다시 그리고, 찜 · 필터 · 검색어 같은 상태가 여러 파일에 흩어지면서 화면과 상태를 맞추는 일이 점점 어려워졌습니다. 반복되는 DOM 조작은 함수로 묶어 버텼지만, 이 불편이 다음 프로젝트에서 "상태가 바뀌면 화면이 따라오는" React를 선택한 이유가 됐습니다.



detail — 공고번호 기반 상세. '앱에서 보기 · 입양신청' 버튼은 화면만 만들어 누르면 준비 중 안내와 보호센터 연락처를 보여 줍니다.



찜 · 로그인 상태 — localStorage에 저장한 찜 목록으로 필터, 헤더에 로그인 사용자 표시

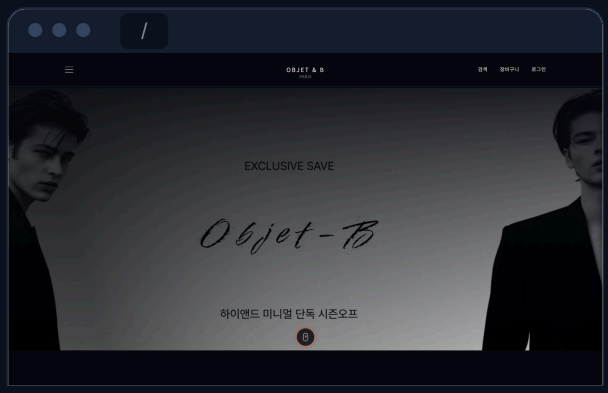


댓글 — 공고번호별로 localStorage에 저장, `textContent`로 렌더링

REACT · TYPESCRIPT

Objet & B

미니멀 쇼핑몰 프로토타입을 React + TypeScript + Firebase로 다시 만들고, 토스페이먼츠 테스트 결제를 서버 금액 검증까지 연결한 개인 프로젝트



기간	역할
2026.07 – 2026.10	1인 개발
배포	**협업 방식**
Vercel · 운영 중	브랜치 → PR → 미리보기 배포

Live GitHub

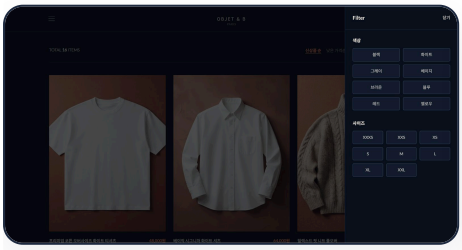
01 기술 스택과 데이터 저장 위치 STACK · DATA

데이터마다 "누가, 얼마나 오래 써야 하는지"를 기준으로 저장 위치를 정했습니다.

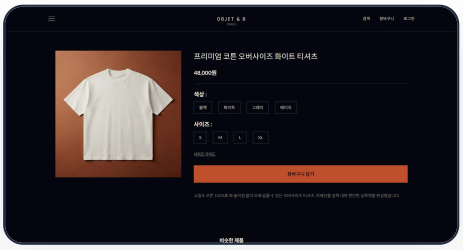
React 19 · TypeScript · Vite · Tailwind CSS v4 · React Router · TanStack Query · Firebase Auth · Firestore · 토스페이먼츠 SDK · Vercel Functions · Vitest · GSAP

데이터	저장 위치	이유
상품	코드 안의 데이터 (`ProductsData.ts`)	자주 바뀌지 않는 포트폴리오 범위라 API 없이 관리
주문	Firestore `orders`	사용자별 저장 · 조회 (결제 승인 후 저장, `paymentKey` 포함)
결제 중인 주문	sessionStorage	결제창에 다녀오는 동안만 필요한 임시 데이터, 탭마다 따로
배송지	Firestore `users/{uid}/addresses`	이름 · 전화번호가 있어 브라우저가 아닌 계정에 저장
장바구니	localStorage `objet-b-cart-{uid}`	결제 전 임시 데이터, 계정별 키로 분리
최근 본 상품	localStorage	이 브라우저에서만 의미 있는 기록

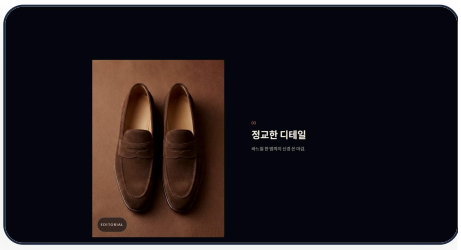
Firestore 보안 규칙으로 본인 uid의 데이터만 읽고 쓸 수 있게 막고, 배송지는 정해진 항목과 글자 수만 저장되도록 제한했습니다.



상품 목록 · 색상 · 사이즈 필터 패널



상품 상세 · 색상 · 사이즈 옵션



브랜드 스토리 · GSAP 핀 고정 · 가로 스크롤

02 결제 흐름 — 브라우저를 믿지 않는 구조 PAYMENT

버튼만 있던 결제를 토스페이먼츠 테스트 결제로 연결했습니다. 금액은 브라우저가 바꿀 수 있으니, 서버(Vercel 함수)가 상품 데이터로 다시 계산해 맞을 때만 승인을 요청합니다. 시크릿 키는 서버에서만 씁니다.

CLIENT · React

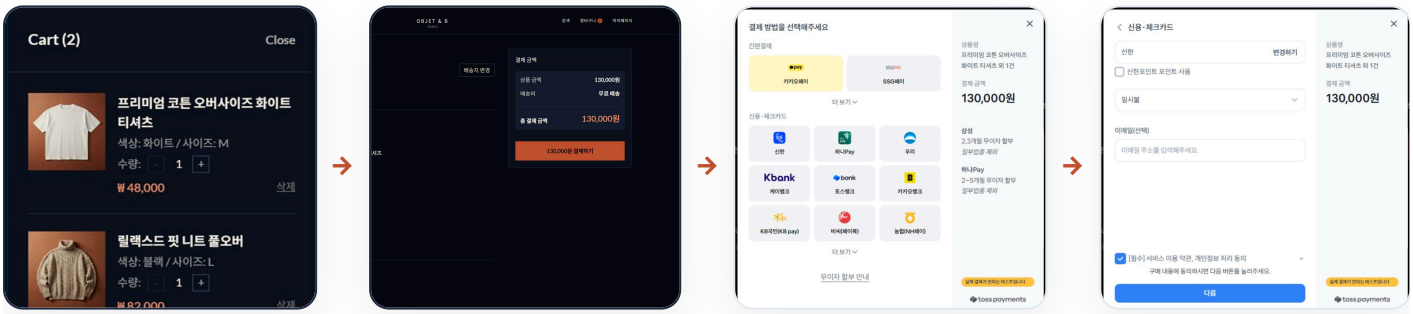
- **1** 주문서에서 결제 요청 → 주문 정보를 sessionStorage에 보관
- **2** 토스 결제창에서 인증
- **3** 성공 주소로 이동 → `/api/confirm`에 `paymentKey · orderId · amount · items` 전송 (가격 대신 "무엇을 몇 개"만)
- **7** 주문을 Firestore에 저장 → 장바구니 비우기

SERVER · Vercel Function

- **4** `calculateAmount(items)`로 금액 재계산
- **4-1** 없는 상품 · 이상한 수량 · 금액 불일치 → **400**, 토스에 승인 요청 안 함 (인증만 된 결제는 자동 취소)
- **5** 시크릿 키로 토스 승인 API 호출
- **5-1** 이미 승인된 결제면 결제를 조회해, 상태 · 주문번호 · 금액이 같을 때 같은 결과로 응답 (중복 요청에도 결과가 같게)
- **6** 화면에 필요한 값만 골라 응답 (결제 비밀 값 · 카드 정보 제외)

EXTERNAL

- **Toss Payments** 결제창 · 승인 API
- **Firebase** Auth · Firestore 보안 규칙



① 장바구니
옵션별 항목 · 총액

② 주문서
배송지 · 결제 금액 확인

③ 토스 결제창
결제 방법 선택

④ 카드 · 약관
다음은 카드사 인증 → 서버 검증

테스트 계정으로 로그인해 배포된 사이트의 토스페이먼츠 테스트 결제창까지 실제로 진행하며 캡처했습니다. 그다음 카드사 앱 인증은 실제 카드가 필요해 진행하지 않았고, 승인 이후 과정은 위 흐름도와 다음 쪽의 코드로 설명했습니다.

03 금액 검증 코드와 테스트 VERIFY · TEST

결제처럼 틀리면 안 되는 로직은 서버와 테스트가 같은 함수를 쓰도록 분리하고, 경계값까지 테스트로 고정했습니다.

src/utils/calculateAmount.ts

```ts
const MAX_QTY = 20;

// 상품 번호와 수량으로 서버가 직접 합계를 계산
// 없는 상품이거나 수량이 이상하면 null
export function calculateAmount(items) {
  let total = 0;
  for (const item of items) {
    const product = PRODUCTS.find((p) => p.id ===
item.productId);
    const isValidQty = Number.isInteger(item.qty)
      && item.qty >= 1 && item.qty <= MAX_QTY;
    if (!product || !isValidQty) return null;
    total += product.price * item.qty;
  }
  return total;
}
```

api/confirm.ts (발췌)

```ts
const expectedAmount = calculateAmount(items);
if (expectedAmount === null) return /* 400 */;
if (expectedAmount !== amount) return /* 400 */;

const secretKey = process.env.TOSS_SECRET_KEY; //
서버 전용
const auth = btoa(`${secretKey}:`);

// 같은 결제를 또 승인하려 한 경우
if (result.code === "ALREADY_PROCESSED_PAYMENT") {
  // 결제를 조회해 DONE · 주문번호 · 금액이 같으면
성공으로
}
```

VITEST

결제 금액 계산 단위 테스트

금액 계산을 서버 함수에서 `src/utils` 로 분리해 서버와 테스트가 같은 함수를 쓰게 했습니다. 경계 조건을 일부러 `<=` 에서 `<` 로 바꿔 테스트가 실제로 실패하는 것까지 확인했습니다.

```
✓ src/utils/calculateAmount.test.ts (8)
  ✓ 1번째 상품이 2개 → 96,000
  ✓ 1번째 1개 + 2번째 1개 → 112,000
  ✓ 없는 상품(999) → null
  ✓ 수량 0개 → null
  ✓ 수량 1.5개 → null
  ✓ 딱 20개 → 960,000
  ✓ 21개 → null (최대 20개 초과)
  ✓ 빈 배열 → 0
```

테스트 이름과 기댓값을 그대로 옮겨 정리한 화면입니다.

04 서버 데이터와 상태 관리 STATE

서버 데이터는 직접 관리하던 로딩 · 에러 상태를 React Query로 옮기고, 장바구니 같은 전역 상태는 Context API로 계정별로 나눴습니다.

TANSTACK QUERY

마이페이지 주문 내역

`useState` · `useEffect` 로 직접 관리하던 로딩 · 취소 깃발 · 예외 처리를 `useQuery` 로 바꿨습니다.

- 조회에 실패해도 "주문 내역이 없습니다"가 보이던 문제 → **에러 안내를 따로 구분**
- 다른 페이지에 갔다 오면 다시 불러오던 것 → **캐시로 바로 표시**

src/Mypage/Mypage.tsx (발췌)

```
const { data, isLoading, isError } = useQuery({
  queryKey: ["orders", user?.uid],
  // ...Firestore에서 내 주문만 최신순 조회
});
```

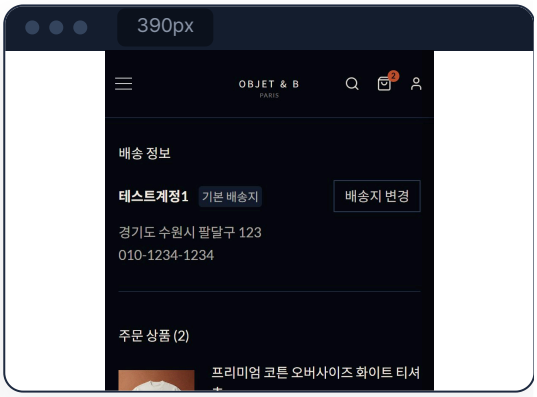
CONTEXT API

계정별 장바구니와 로그인 시 병합

장바구니가 한 브라우저의 모든 계정에서 공유되던 문제를, 계정별 키(`objet-b-cart-{uid}`)로 분리해 고쳤습니다. 비로그인 상태에서 담은 상품은 로그인할 때 합쳐 주고(같은 상품 · 옵션은 수량을 더하되 최대 수량 제한), 로그인 · 로그아웃으로 키가 바뀌는 순간 이전 목록이 새 키에 잘못 저장되지 않도록 "지금 목록이 어느 키에서 왔는지"를 따로 기억합니다.



마이페이지 — 주문이 없을 때는 빈 안내, 조회에 실패하면 별도 에러 문구 · 최근 본 상품 (localStorage)



모바일 주문서 — 헤더는 아이콘, 장바구니 수는 아이콘 위 뱃지로

05 개발 방식 WORKFLOW

브랜치 → PR → 머지

2026년 10월부터 기능마다 브랜치를 만들고, PR에 "왜 · 무엇을 · 확인"을 남긴 뒤 머지했습니다.

머지 전 미리보기 배포

PR마다 생기는 Vercel 미리보기 배포에서 실제 서버 환경을 먼저 확인합니다. 배포에서만 나는 500 에러를 이렇게 잡았습니다.

타입 검사 · 테스트 · 빌드

`npm run typecheck` 가 `src/` 와 `api/` 를 함께 검사하고, `npm test` 로 금액 계산을 확인합니다.

06 트러블슈팅 TROUBLESHOOTING

결제 승인을 서버 함수로 붙이면서는 코드보다 개발 · 배포 환경 설정에서, 배포 후에는 실기기와 뒤로 가기 같은 실제 사용 흐름에서 문제를 찾았습니다.

DEPLOY · ES MODULE

배포한 결제 서버 함수만 500

원인 로그의 `ERR_MODULE_NOT_FOUND`. ES 모듈 환경의 Node는 import 경로에 확장자가 없으면 파일을 못 찾는데, 로컬은 개발 도구가 대신 찾아 줘서 드러나지 않음

해결 서버 함수가 불러오는 import 연결 고리 전체에 `.js` 확장자. 머지 전 미리보기 배포에서 발견

기술 블로그에 자세한 과정 →

BROWSER · BFCACHE

결제 후 뒤로 가기 → 끝난 결제창에 갇힘

원인 bfcache가 결제창이 떠 있던 주문서를 그대로 얼려 뒀다가 복원. `pageshow`의 `persisted: true`로 확인

해결 복원을 감지하면 새로고침. 완료 페이지에서 뒤로 가기를 막는 방식은 다른 경로를 못 막아 고르지 않음

기술 블로그에 자세한 과정 →

REACT · CLOSURE

결제창이 떠 있는 채로 뒤로 가기 → 창이 남음

원인 토스 SDK가 결제창을 React 바깥(body)에 붙여 화면 전환 후에도 남음. `[]` effect의 cleanup은 첫 렌더링의 state 값을 기억(클로저)

해결 주문서가 사라질 때 결제 중이면 정리 후 새로고침. 결제 중 여부는 `useRef`로 최신 값을 읽음

기술 블로그에 자세한 과정 →

LOCAL DEV · VERCEL

`vercel dev`로 띄우면 백지 화면

원인 SPA 새로고침용 "모든 경로 → index.html" 규칙이 `/src/main.tsx` 같은 개발 서버 요청까지 가로챔. 배포에선 실제 파일이 먼저 처리돼 안 보였음

해결 규칙을 페이지 주소에만 적용되게 `/((?!api/|@)[^.]*)`로 좁힘

iOS SAFARI

아이폰에서만 생기는 좌우 스크롤

원인 16px보다 작은 입력칸을 누르면 Safari가 화면을 확대한 채로 둠. PC 기기 모드로는 재현 안 됨

해결 입력칸을 휴대폰 폭에서 16px로. 손가락 확대까지 막는 `maximum-scale=1`은 접근성 때문에 쓰지 않음

PAYMENT SDK

PC에서 결제창을 닫아도 실패 페이지로 안 감

원인 PC 결제창은 페이지 위에 겹쳐 떠서, 닫으면 주소 이동 없이 `USER_CANCEL` 에러로 끝남

해결 취소는 오류가 아니라 사용자의 선택이라, 오류 알림 대신 "장바구니와 배송지는 그대로 있어요" 안내만

이 밖에 TypeScript 7의 타입 범위(`api/tsconfig.json` 분리), Windows에서만 통과하던 대소문자 다른 import, 로컬로 내려오지 않는 Secret 환경변수도 README에 정리했습니다.

임상빈
Frontend Developer

010-5150-5589 · persie24@naver.com
포트폴리오 sangbin-dev.vercel.app · 기술 블로그 velog.io/@bin-dev
포인핸드 Live · GitHub | Objet & B Live · GitHub